Consultas y subconsultas

SQL · Segunda parte | Resumen con consultas y resultados

> **Cómo leer los ejemplos:** las consultas siguen los temas del apunte. Los datos son pequeños ejemplos didácticos adaptados; no reproducen todas las filas de sus capturas. Los resultados se calcularon sobre estas tablas. Sin ORDER BY, el orden de las filas no está garantizado.

1. ORDER BY · Ordenar el resultado

Se escribe al final de la consulta. **ASC** ordena de menor a mayor y es el valor por defecto; **DESC**, de mayor a menor. Si hay varias columnas, la prioridad se lee de izquierda a derecha.

```
SELECT Mat, Anio, Leg
FROM Inscripcion
ORDER BY Anio ASC, Mat DESC;
```

Mat	Anio	Leg
1002	2016	109
1001	2016	69
1005	2017	109
1002	2017	138
1001	2017	109

Primero se ordena por año; dentro de cada año, por materia descendente. Solo si el primer criterio empata se aplica el segundo.

2. Alias de tabla · Abreviar y distinguir columnas

```
SELECT A.Leg, A.Nom, A.Ape
FROM Alumno AS A;
```

A es el alias de Alumno. También podés escribir **FROM Alumno A**. El punto indica a qué tabla pertenece una columna: A.Leg. Es especialmente útil cuando varias tablas tienen columnas con el mismo nombre.

Datos de referencia · Alumnos e inscripciones

La tabla Inscripcion es la mostrada arriba. Esta es la tabla Alumno, que usaremos en las subconsultas de las páginas 4 y 5:

Leg	Nom	Ape	Cuota
69	Juan	Martínez	500
109	Pedro	García	550
138	Pablo	Pérez	600
160	Silvana	Gómez	700

> **Para armar una consulta:** identificá qué tablas necesitás, cómo se relacionan y qué filas querés conservar. Después elegí las columnas y, si corresponde, el orden del resultado.

Fuente: Universidad de Palermo, 4G0016_APU_Consultas _Subconsultas9_2023.pdf, pp. 1-16. Resumen reformulado con ejemplos adaptados.

Juntar tablas · JOIN

Coincidencias y conservación de filas | Ejemplos con profesores y cursos

Un JOIN combina filas mediante una condición escrita en **ON**. Las columnas relacionadas pueden tener nombres distintos; lo importante es vincular los datos correctos, normalmente una FK con su PK.

Tablas de partida

Prof	NyA
1	Juan García
2	José Pérez
3	Marta Báez

Mat	Anio	Prof
1001	2016	1
1002	2017	2
1005	2018	NULL

3. INNER JOIN · Solo las coincidencias

Muestra las combinaciones que cumplen la condición de junta. Marta no aparece porque no tiene curso; el curso 1005 tampoco aparece porque no tiene profesor.

```
SELECT P.NyA, C.Mat, C.Anio
FROM Profesor P INNER JOIN Curso C
  ON P.Prof = C.Prof;
```

NyA	Mat	Anio
Juan García	1001	2016
José Pérez	1002	2017

4. LEFT JOIN · Todas las filas de la izquierda

Conserva todos los profesores. Si alguno no tiene curso, completa con **NULL** las columnas correspondientes a Curso.

```
SELECT P.NyA, C.Mat, C.Anio
FROM Profesor P LEFT JOIN Curso C
  ON P.Prof = C.Prof;
```

NyA	Mat	Anio
Juan García	1001	2016
José Pérez	1002	2017
Marta Báez	NULL	NULL

Clave: el lado izquierdo es la tabla anterior a LEFT JOIN. Una misma fila puede aparecer varias veces si tiene varias coincidencias en la otra tabla.

Más tipos de JOIN

RIGHT JOIN, FULL JOIN y consultas con varias tablas

5. RIGHT JOIN · Todas las filas de la derecha

Conserva todos los cursos, incluso los que no tienen profesor asignado. En esos casos, los datos del profesor aparecen como NULL.

```
SELECT P.NyA, C.Mat, C.Anio
FROM Profesor P RIGHT JOIN Curso C
  ON P.Prof = C.Prof;
```

NyA	Mat	Anio
Juan García	1001	2016
José Pérez	1002	2017
NULL	1005	2018

6. FULL JOIN · Todas las filas de ambos lados

Incluye coincidencias, profesores sin curso y cursos sin profesor. Completa con NULL las columnas del lado que no tiene correspondencia.

```
SELECT P.NyA, C.Mat, C.Anio
FROM Profesor P FULL JOIN Curso C
  ON P.Prof = C.Prof;
```

NyA	Mat	Anio
Juan García	1001	2016
José Pérez	1002	2017
Marta Báez	NULL	NULL
NULL	1005	2018

7. Tres o más tablas · Agregar un JOIN por vínculo

Para buscar clientes que compraron artículos del rubro 3, se sigue la relación **Cliente → Factura → Detalle → Articulo**:

```
SELECT DISTINCT C.Nrocli, C.Nyape
FROM Cliente C
INNER JOIN Factura F ON C.Nrocli = F.Cliente
INNER JOIN Detalle D ON F.Nrofactura = D.Nrofactura
INNER JOIN Articulo A ON D.Articulo = A.NroArtic
WHERE A.Rubro = 3;
```

Mini ejemplo: Ana López (cliente 1) tiene la factura 10 con dos artículos del rubro 3. Luis Díaz (cliente 2) tiene la factura 20 con un artículo del rubro 4. El resultado es:

Nrocli	Nyape
1	Ana López

DISTINCT: evita repetir a Ana por sus dos artículos del rubro 3. ON relaciona tablas; WHERE filtra. En un JOIN externo, filtrar en WHERE columnas del lado sin coincidencia puede eliminar las filas completadas con NULL.

IN y subconsultas

Comparar con un conjunto o con un único valor

8. IN y NOT IN · Pertenecer a una lista

IN verifica si un valor pertenece al conjunto; **NOT IN**, si queda fuera. Para esta sección usamos la siguiente tabla Materia:

Mat	Nombre
1001	Matemática
1002	Programación
1005	Redes
1009	Inglés

```
SELECT *
FROM Materia
WHERE Mat IN (1001, 1002, 1005);
```

Resultado: Matemática (1001), Programación (1002) y Redes (1005). Con **NOT IN** sobre la misma lista, el resultado es Inglés (1009). IN equivale aquí a unir las igualdades con OR.

9. IN con subconsulta · El conjunto viene de otra consulta

Para listar alumnos inscriptos en 2017, primero se obtiene el conjunto de legajos de ese año y luego se buscan esos alumnos:

```
SELECT A.Leg, A.Nom, A.Ape
FROM Alumno A
WHERE A.Leg IN (SELECT I.Leg
                FROM Inscripcion I
                WHERE I.Anio = 2017);
```

La subconsulta devuelve los legajos **109, 138 y 109**. La repetición del 109 no duplica al alumno en la consulta externa. Resultado:

Leg	Nom	Ape
109	Pedro	García
138	Pablo	Pérez

En este uso, la subconsulta debe devolver **una columna** con valores de un tipo compatible con A.Leg. Con NOT IN, y con estos datos sin legajos nulos, el resultado sería Juan (69) y Silvana (160).

10. Comparación con subconsulta · Un solo valor

Alumnos con cuota superior a la del legajo 69:

```
SELECT A.Leg, A.Nom, A.Cuota
FROM Alumno A
WHERE A.Cuota > (SELECT AL.Cuota
                 FROM Alumno AL
                 WHERE AL.Leg = 69);
```

Leg	Nom	Cuota
109	Pedro	550
138	Pablo	600
160	Silvana	700

La subconsulta devuelve **500**. Con >, <, =, >=, <= o <>, una subconsulta escalar debe dar un único valor: más de una fila produce error; si no devuelve filas, su valor es NULL.

EXISTS y repaso

Comprobar existencia y evitar errores frecuentes

11. EXISTS · ¿Hay alguna fila relacionada?

Muestra cada alumno para el que existe al menos una inscripción en 2017. La condición **I.Leg = A.Leg** vincula la subconsulta con el alumno que se está evaluando.

```
SELECT A.Leg, A.Nom, A.Ape
FROM Alumno A
WHERE EXISTS (SELECT 1
              FROM Inscripcion I
              WHERE I.Leg = A.Leg
                AND I.Anio = 2017);
```

Leg	Nom	Ape
109	Pedro	García
138	Pablo	Pérez

SELECT 1 no cuenta ni devuelve el legajo: EXISTS solo verifica si hay alguna fila. Para esta prueba, SELECT * también sirve. Como con IN, las varias inscripciones de Pedro no lo duplican.

12. NOT EXISTS · ¿No hay ninguna fila relacionada?

Para listar alumnos que no se inscribieron en 2017:

```
SELECT A.Leg, A.Nom, A.Ape
FROM Alumno A
WHERE NOT EXISTS (SELECT 1
                  FROM Inscripcion I
                  WHERE I.Leg = A.Leg
                    AND I.Anio = 2017);
```

Leg	Nom	Ape
69	Juan	Martínez
160	Silvana	Gómez

No es lo mismo que WHERE I.Anio <> 2017: Pedro se inscribió en 2016 y también en 2017. Filtrar otro año lo incluiría; NOT EXISTS lo excluye porque sí tiene una inscripción en 2017. Silvana, que nunca se inscribió, también debe aparecer.

Ojo con NOT IN y NULL: si la subconsulta devuelve algún NULL, NOT IN puede no devolver las filas esperadas. Para buscar ausencia de filas relacionadas, NOT EXISTS evita ese problema. La escritura correcta es NOT EXISTS, con S final.

Elegí la herramienta según lo que te piden

Necesitás…	Usás…
Ordenar filas	ORDER BY
Combinar columnas de tablas relacionadas	JOIN + ON
Conservar filas sin coincidencia	LEFT / RIGHT / FULL JOIN
Comparar con una lista o conjunto	IN / NOT IN
Comparar con un único dato calculado por consulta	Operador + subconsulta escalar
Comprobar si hay o no filas relacionadas	EXISTS / NOT EXISTS

JOIN vs. IN / EXISTS: un JOIN puede repetir alumnos cuando hay varias inscripciones. Para obtener una sola fila por alumno con JOIN, podés necesitar DISTINCT. IN y EXISTS filtran las filas de Alumno sin multiplicarlas.